

Họ và tên:

Mã số Sinh viên:

Lớp:

HỆ ĐIỀU HÀNH BÁO CÁO LAB 6

TÓM TẮT BÁO CÁO

1. Bài tập thực hành

	6.4.1	6.4.2	6.4.3	6.4.4	6.4.5
Phương pháp	☐	☐	☐	☐	☐
Minh chứng	☐	☐	☐	☐	☐
Giải thích	☐	☐	☐	☐	☐

Tự đánh giá:

**Lưu ý: Xuất file PDF, đặt tên theo cú pháp:*

<MSSV>_LABx.pdf

REPORT

I. Bài tập thực hành

- Mã nguồn của chương trình:

```
1 #include <sys/wait.h>
2
3 #include <err.h>
4 #include <fcntl.h>
5 #include <signal.h>
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9 #include <unistd.h>
10
11 #define HISTORY_DEFSIZE 32
12
13 typedef struct {
14     size_t allocated, count, index;
15     char **entries;
16 } History;
17
18 int history_add(History *, const char *);
19 void history_free(History *);
20 int history_init(History *, size_t);
21 char *history_parseback(History *);
22
23 void free_arguments(char **);
24 char **split_arguments(char *);
25 void handle_io_redirects(char **, int, int);
26 void parse_and_execute(char **);
27 void prompt(void);
28 void signal_handler(int);
29
30 int
31 main(void)
32 {
33     History hist;
34
35     if (history_init(&hist, HISTORY_DEFSIZE) == 0)
36         errx(1, "cannot initialize history");
37     signal(SIGINT, signal_handler);
38
39     for (;;) {
```

2,0-1

Top

```

26 >         >         size_t linecap = 0;
25 >         >         char **args, *line = NULL;
24
23 >         >         prompt();
22
21 >         >         if (getline(&line, &linecap, stdin) == -1) {
20 >         >         >         printf("\n");
19 >         >         >         free(line);
18 >         >         >         break;
17 >         >         }
16 >         >         line[strlen(line) - 1] = '\0';
15 >         >         if ((args = split_arguments(line)) == NULL) {
14 >         >         >         free(line);
13 >         >         >         continue;
12 >         >         }
11 >         >         if (args[0] == NULL) {
10 >         >         >         free_arguments(args);
9 >         >         >         free(line);
8 >         >         >         continue;
7 >         >         } else if (strcmp(args[0], "exit") == 0) {
6 >         >         >         free_arguments(args);
5 >         >         >         free(line);
4 >         >         >         break;
3 >         >         } else if (strcmp(args[0], "HF") == 0) {
2 >         >         >         free_arguments(args);
1 >         >         >         free(line);
66 |
1 >         >         >         if ((line = history_parseback(&hist)) == NULL) {
2 >         >         >         >         warnx("HF: cannot fetch backward");
3 >         >         >         >         continue;
4 >         >         >         }
5 >         >         >         if ((args = split_arguments(line)) == NULL) {
6 >         >         >         >         free(line);
7 >         >         >         >         continue;
8 >         >         >         }
9 >         >         } else if (history_add(&hist, line) == 0)
10 >         >         >         warnx("cannot add command to history\n");
11
12 >         >         parse_and_execute(args);

```

66,0-1

11%

```

28 >         free_arguments(args);
27 >         free(line);
26 >     }
25
24 >     history_free(&hist);
23 >     return 0;
22 }
21
20 void
19 free_arguments(char **args)
18 {
17 >     for (size_t i = 0; args[i] != NULL; i++)
16 >         free(args[i]);
15 >     free(args);
14 }
13
12 char **
11 split_arguments(char *line)
10 {
9 >     size_t index = 0, maxargs = strlen(line) / 2 + 2;
8 >     char **args, *dupline, *token;
7
6 >     if ((args = malloc(maxargs * sizeof(char *))) == NULL)
5 >         return NULL;
4 >     if ((dupline = strdup(line)) == NULL) {
3 >         free(args);
2 >         return NULL;
1 >     }
107 █
1 >     token = strtok(dupline, " \t");
2 >     while (token != NULL) {
3 >         if ((args[index++] = strdup(token)) == NULL) {
4 >             args[index] = NULL;
5 >             free_arguments(args);
6 >             free(dupline);
7 >             return NULL;
8 >         }
9 >         token = strtok(NULL, " \t");
10 >     }

```

107,0-1

23%

```

25 >     args[index] = NULL;
24 >     free(dupline);
23 >     return args;
22 }
21
20 void
19 handle_io_redirects(char **args, int isfirstcmd, int islastcmd)
18 {
17 >     for (size_t i = 0; args[i] != NULL; i++) {
16 >         >     int fd, flags, mode, target;
15
14 >         >     if (strcmp(args[i], "<") == 0) {
13 >             >         if (isfirstcmd == 0)
12 >             >             >         errx(1, "invalid redirection");
11 >             >             >         flags = O_RDONLY;
10 >             >             >         mode = 0;
9 >             >             >         target = STDIN_FILENO;
8 >             >         } else if (strcmp(args[i], ">") == 0) {
7 >             >             >         if (islastcmd == 0)
6 >             >             >             >         errx(1, "invalid redirection");
5 >             >             >             >         flags = O_CREAT | O_TRUNC | O_WRONLY;
4 >             >             >             >         mode = 0644;
3 >             >             >             >         target = STDOUT_FILENO;
2 >             >         } else
1 >             >             >         continue;
143 |
1 >         >         if (args[i + 1] == NULL || strcmp(args[i + 1], "<") == 0 ||
2 >             >             strcmp(args[i + 1], ">") == 0 ||
3 >             >             strcmp(args[i + 1], "|") == 0)
4 >             >             >         errx(1, "invalid argument after %s", args[i]);
5 >             >         if ((fd = open(args[i + 1], flags, mode)) == -1)
6 >             >             >         err(1, "%s", args[i + 1]);
7 >             >         if (dup2(fd, target) == -1)
8 >             >             >         err(1, "dup2");
9 >             >         close(fd);
10
11 >             >         free(args[i]);
12 >             >         free(args[i + 1]);
13 >             >         size_t j = i;

```

143,0-1

35%

```

157 >         for (; args[j + 2] != NULL; j++)
1 >             args[j] = args[j + 2];
2 >             args[j] = NULL;
3 >             args[j + 1] = NULL;
4 >             i--;
5 >     }
6 }
7
8 int
9 history_add(History *hist, const char *line)
10 {
11 >     if (hist->count >= hist->allocated) {
12 >         size_t tsize = hist->allocated * 2;
13 >         char **backup = hist->entries;
14 >         char **tptr = realloc(hist->entries, tsize * sizeof(char *));
15 >         if (tptr == NULL) {
16 >             hist->entries = backup;
17 >             return 0;
18 >         }
19 >         hist->allocated = tsize;
20 >         hist->entries = tptr;
21 >     }
22
23 >     if ((hist->entries[hist->count] = strdup(line)) == NULL)
24 >         return 0;
25 >     hist->index = ++hist->count;
26 >     return 1;
27 }
28
29 void
30 history_free(History *hist)
31 {
32 >     for (size_t i = 0; i < hist->count; i++)
33 >         free(hist->entries[i]);
34 >     free(hist->entries);
35
36 >     hist->allocated = 0;
37 >     hist->count = 0;
38 >     hist->index = 0;

```

157,2-9

47%


```

24 }
23
22 int
21 history_init(History *hist, size_t size)
20 {
19     if (size == 0)
18     >         return 0;
17     hist->entries = malloc(size * sizeof(char *));
16     if (hist->entries == NULL)
15     >         return 0;
14
13     hist->allocated = size;
12     hist->count = 0;
11     hist->index = 0;
10     return 1;
9 }
8
7 char *
6 history_parseback(History *hist)
5 {
4     if (hist->count == 0 || hist->index == 0)
3     >         return NULL;
2     return strdup(hist->entries[--hist->index]);
1 }
220
1 void
2 parse_and_execute(char **args)
3 {
4     size_t pcount = 0;
5     for (size_t i = 0; args[i] != NULL; i++) {
6     >         if (strcmp(args[i], "|") != 0)
7     >             continue;
8
9     >         if (i == 0 || args[i + 1] == NULL ||
10    >             strcmp(args[i + 1], "<") == 0 ||
11    >             strcmp(args[i + 1], ">") == 0 ||
12    >             strcmp(args[i + 1], "|") == 0) {
13    >             warnx("invalid piping");
14    >             return;

```

220,0-1

59%

```

19 >     >     }
18 >     >     pcount++;
17 >     > }
16
15 >     if (pcount == 0) {
14 >         >     switch (fork()) {
13 >         >     case -1:
12 >         >         >     perror("fork");
11 >         >         >     return;
10 >         >     case 0:
9 >         >         >     signal(SIGINT, SIG_DFL);
8 >         >         >     handle_io_redirects(args, 1, 1);
7 >         >         >     execvp(args[0], args);
6 >         >         >     err(1, "%s", args[0]);
5 >         >     default:
4 >         >         >     wait(NULL);
3 >         >     }
2 >         >     return;
1 >     > }
254
1 >     pid_t *pids = calloc(pcount + 1, sizeof(pid_t));
2 >     if (pids == NULL) {
3 >         >     perror("calloc");
4 >         >     return;
5 >     }
6 >     char ***cmds = calloc(pcount + 1, sizeof(char **));
7 >     if (cmds == NULL) {
8 >         >     perror("calloc");
9 >         >     free(pids);
10 >         >     return;
11 >     }
12 >     size_t start = 0, index = 0;
13 >     for (size_t end = 0;; end++) {
14 >         >     if (args[end] != NULL && strcmp(args[end], "|") != 0)
15 >         >         >     continue;
16
17 >         >     size_t size = end - start;
18 >         >     cmds[index] = malloc((size + 1) * sizeof(char *));
19 >         >     if (cmds[index] == NULL) {

```

254,0-1

71%


```

24 >         >         perror("malloc");
23 >         >         goto cleanup;
22 >         >     }
21
20 >         >     for (size_t i = 0; i < size; i++)
19 >         >         >         cmds[index][i] = args[start + i];
18 >         >         cmds[index++][size] = NULL;
17 >         >         start = end + 1;
16 >         >         if (args[end] == NULL)
15 >         >         >         break;
14 >     }
13
12 >     for (size_t i = 0; i <= pcount; i++) {
11 >         >     for (size_t j = 0; cmds[i][j] != NULL; j++) {
10 >         >         >     if (strcmp(cmds[i][j], "<") == 0 && i != 0) {
9 >         >         >         >     warnx("invalid redirection");
8 >         >         >         >     goto cleanup;
7 >         >         >     }
6 >         >         >     if (strcmp(cmds[i][j], ">") == 0 && i != pcount) {
5 >         >         >         >     warnx("invalid redirection");
4 >         >         >         >     goto cleanup;
3 >         >         >     }
2 >         >     }
1 >     }
298 |
1 >     int prev_outpipe = -1;
2 >     for (size_t i = 0; i <= pcount; i++) {
3 >         >     int fildes[2] = { -1, -1 };
4 >         >     if (i < pcount && pipe(fildes) == -1) {
5 >         >         >     perror("pipe");
6 >         >         >     goto cleanup;
7 >         >     }
8
9 >         >     pids[i] = fork();
10 >         >     switch (pids[i]) {
11 >         >     case -1:
12 >         >         >     close(fildes[0]);
13 >         >         >     close(fildes[1]);
14 >         >         >     perror("fork");

```

298,0-1

82%

```

33 >         >         goto cleanup;
32 >         >         case 0:
31 >         >         >         signal(SIGINT, SIG_DFL);
30
29 >         >         >         if (i == 0)
28 >         >         >         >         handle_io_redirects(cmds[i], 1, 0);
27 >         >         >         else {
26 >         >         >         >         if (dup2(prev_outpipe, STDIN_FILENO) == -1)
25 >         >         >         >         >         err(1, "dup2");
24 >         >         >         >         }
23 >         >         >         if (i == pcount)
22 >         >         >         >         handle_io_redirects(cmds[i], 0, 1);
21 >         >         >         else {
20 >         >         >         >         if (dup2(fildes[1], STDOUT_FILENO) == -1)
19 >         >         >         >         >         err(1, "dup2");
18 >         >         >         >         }
17
16 >         >         >         if (prev_outpipe != -1)
15 >         >         >         >         close(prev_outpipe);
14 >         >         >         close(fildes[0]);
13 >         >         >         close(fildes[1]);
12
11 >         >         >         execvp(cmds[i][0], cmds[i]);
10 >         >         >         err(1, "%s", cmds[i][0]);
9 >         >         default:
8 >         >         >         if (prev_outpipe != -1)
7 >         >         >         >         close(prev_outpipe);
6 >         >         >         if (i < pcount) {
5 >         >         >         >         close(fildes[1]);
4 >         >         >         >         prev_outpipe = fildes[0];
3 >         >         >         }
2 >         >         }
1 >         }
346 █
1 >         for (size_t i = 0; i <= pcount; i++)
2 >         >         waitpid(pids[i], NULL, 0);
3 cleanup:
4 >         for (size_t i = 0; cmds[i] != NULL && i <= pcount; i++)
5 >         >         free(cmds[i]);

```

346,0-1

94%

```

10 >     free(cmds);
9 >     free(pids);
8 }
7
6 void
5 prompt(void)
4 {
3 >     printf("it007sh> ");
2 >     fflush(stdout);
1 }
362 █
1 void
2 signal_handler(int __attribute__((unused)) signal)
3 {
4 >     printf("\n");
5 }
~
~

```

1. Thực thi command trong tiến trình con

- **Phương pháp:** dùng hàm `getline()` đọc chuỗi nhập vào, sau đó tách chuỗi ra thành mảng các chuỗi, dùng `fork()` tạo tiến trình con rồi dùng hàm `execvp()` thực thi lệnh từ mảng các chuỗi trong tiến trình con.
- **Minh chứng:**

```

it007sh> ifconfig enp0s31f6
enp0s31f6: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
        ether 6c:24:08:fc:28:4e txqueuelen 1000 (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
        device interrupt 16  memory 0xae400000-ae420000

it007sh> █

```

- **Giải thích:** sau khi người dùng nhấn Enter, chuỗi nhận được từ `getline()` có dạng “ifconfig enp0s31f6\n\n0”. Chuyển ký tự “\n” thành “\0” rồi dùng hàm `split_arguments()` chuyển thành mảng `args = { “ifconfig”, “enp0s31f6”, “NULL” }`. Hàm `split_arguments()` dùng `strtok()` để tách phần. Hàm `execute(args)` tạo tiến trình con bằng `fork()`, ở tiến trình

con thực thi lệnh bằng hàm `execvp(args[0], args)`, ở tiến trình cha có `wait(NULL)` đợi tiến trình con thực thi xong mới tiếp tục vòng lặp, in prompt mới và nhận input.

2. Tạo tính năng sử dụng lại câu lệnh gần đây nhất:

Cung cấp History Feature, cho phép người dùng thực thi lệnh gần đây nhất bằng cách nhập 'HF' và nhấn Enter.

- **Phương pháp:** lưu các lệnh không phải HF vào một mảng riêng, giữ index bằng số phần tử trong mảng. Khi có lệnh HF, giảm index xuống 1 rồi lấy lệnh ở vị trí index thực thi. Khi có lệnh khác, thêm lệnh đó vào mảng, gán lại index bằng với số phần tử trong mảng.

- **Minh chứng:**

```
it007sh> echo abc
abc
it007sh> ls -l
total 19
-rwxr-xr-x 1 tch users 9512 Dec 23 22:56 it007sh
-rw-r--r-- 1 tch users 134 Dec 23 22:41 makefile
-rw-r--r-- 1 tch users 7243 Dec 23 22:51 sh.c
it007sh> pwd
/home/tch/build/tmp/shell
it007sh> HF
/home/tch/build/tmp/shell
it007sh> HF
total 19
-rwxr-xr-x 1 tch users 9512 Dec 23 22:56 it007sh
-rw-r--r-- 1 tch users 134 Dec 23 22:41 makefile
-rw-r--r-- 1 tch users 7243 Dec 23 22:51 sh.c
it007sh> HF
abc
it007sh> █
```

- **Giải thích:** định nghĩa struct tên History gồm bốn thành phần: `allocated`: số phần tử đã được cấp phát; `count`: số phần tử được lưu; `index`: số thứ tự của phần tử kế tiếp phần tử hiện tại đang được trở tới, `entries`: mảng động, dùng để lưu các chuỗi, và bốn hàm thành phần: `history_init()` khởi tạo với số phần tử ban đầu cho trước, `history_add()` thêm phần tử, `history_parseback()` trả về địa chỉ chuỗi trước, `history_free()` giải phóng bộ nhớ.

Đầu chương trình, khởi tạo một History tên *hist* với số phần tử ban đầu 32. Sau đó, sau khi chạy hàm `split_arguments()`, kiểm tra nếu phần tử đầu của `args` không phải `NULL`, `HF` hay `exit` thì dùng `history_add()` thêm vào *hist*. Nếu là `HF`, chương trình giải phóng *line* và *args* hiện tại, rồi dùng `history_parseback()` lấy chuỗi được thêm gần nhất từ *hist*, sau đó tiếp tục quy trình thực thi. Khi kết thúc, `history_free()` giải phóng *hist-entries* trước khi thoát.

3. Hỗ trợ các toán tử chuyển hướng '>' và '<', trong đó '>' chuyển hướng đầu ra của lệnh sang một tệp và '<' chuyển hướng đầu vào của lệnh từ một tệp.

- **Phương pháp:** kiểm tra từng chuỗi trong `args`, nếu là `<` hay `>` thì tạo file descriptor tương ứng rồi chuyển `stdin/stdout` tới file descriptor vừa tạo.

- **Minh chứng:**

```
it007sh> ls
it007sh makefile out.txt sh.c
it007sh> ls > out.txt
it007sh> cat out.txt
it007sh
makefile
out.txt
sh.c
it007sh> sort -R < out.txt
sh.c
it007sh
out.txt
makefile
it007sh> 
```

- **Giải thích:** tạo hàm `handle_io_redirects()` phục vụ `<` và `>`. Trong hàm, duyệt toàn bộ *args* tới khi gặp phần tử `NULL` (kết thúc). Khi gặp, gán vào *flags*, *mode* và *target* các giá trị tương ứng:

- + Khi gặp `<`: *flags* = `O_RDONLY`, *mode* = 0, *target* = `STDIN_FILENO`
- + Khi gặp `>`: *flags* = `O_CREAT` | `O_TRUNC` | `O_WRONLY`, *mode* = 0644, *target* = `STDOUT_FILENO`

Sau đó cho kiểm tra chuỗi sau dấu có hợp lệ (tức không phải '<', '>' hay '|') không. Nếu không cho in lỗi và trả về prompt, nếu có thì dùng lệnh `open()` tạo fd với *flags, mode* ở trên, dùng `dup2()` lấy fd vừa tạo làm đầu ra/vào theo *target* ở trên rồi đóng file descriptor gốc, tiến hành xóa chuỗi chứa dấu '<', '>' và chuỗi chứa nguồn/đích sau nó. Hàm được dùng ngay trước hàm `execvp()` của tiến trình con.

4. Cho phép đầu ra của một lệnh đóng vai trò là đầu vào cho lệnh khác bằng cách sử dụng một đường ống.

- **Phương pháp:** tạo *prev_outpipe* lưu đầu đọc của lệnh trước đó. Với mỗi lệnh không phải lệnh cuối, dùng `pipe()` tạo đường ống nối, dùng `dup2()` lấy *prev_outpipe* (nếu có) làm đầu vào của lệnh, đầu ghi hiện tại thành đầu ra của lệnh rồi đóng các đường sau khi đã dùng xong. Ở tiến trình cha, lưu *prev_outpipe* lại để dùng cho lệnh sau.

- **Minh chứng:**

```
it007sh> ps -aux | grep firefox
tch      2861 18.0  3.2 7096304 800644 ?        S1   Dec23   18:09 /usr/lib/firefox/firefox
tch      2940  0.0  0.2 303524 51572 ?        S1   Dec23    0:02 /usr/lib/firefox/firefox
tch      2968  0.0  0.9 2945972 234804 ?        S1   Dec23    0:03 /usr/lib/firefox/firefox
tch      2974  4.6  0.4 638052 100804 ?        S1   Dec23    4:42 /usr/lib/firefox/firefox
tch      3027  0.4  1.1 23620548 286352 ?       S1   Dec23    0:25 /usr/lib/firefox/firefox
tch      3093  2.1  0.2 459888 62888 ?        S1   Dec23    2:10 /usr/lib/firefox/firefox
tch      3139  0.2  1.1 2819356 276100 ?       S1   Dec23    0:16 /usr/lib/firefox/firefox
tch      3300 14.9  3.1 5590272 763328 ?       S1   Dec23   15:02 /usr/lib/firefox/firefox
tch      8656  1.6  1.8 3264620 443980 ?       S1   Dec23    1:34 /usr/lib/firefox/firefox
tch     16109  0.0  0.0  67660  2996 pts/2    S+   00:01    0:00 grep firefox
tch     22791  0.0  1.0 2877448 257956 ?       S1   Dec23    0:03 /usr/lib/firefox/firefox
tch     24880  8.3  2.0 3285100 496524 ?       S1   Dec23    1:23 /usr/lib/firefox/firefox
tch     26693  0.0  0.5 2908220 131216 ?       S1   Dec23    0:00 /usr/lib/firefox/firefox
tch     26696  0.0  0.5 2842684 131536 ?       S1   Dec23    0:00 /usr/lib/firefox/firefox
tch     27995  0.0  0.5 2712412 131984 ?       S1   Dec23    0:00 /usr/lib/firefox/firefox
tch     29749  0.0  0.6 2783612 150020 ?       S1   Dec23    0:00 /usr/lib/firefox/firefox
it007sh> █
```

- **Giải thích:** đầu tiên, duyệt qua *args* để đếm số '|' và kiểm tra tính hợp lệ (không đứng đầu, đứng cuối hoặc trước '<', '>' hay '|'). Nếu không hợp lệ, cho dừng, báo lỗi rồi trả về prompt. Nếu số lượng '|' là 0, cho thực thi một lệnh như bình thường. Nếu lớn hơn, tạo mảng *cmds* chứa từng lệnh con trong pipeline và *pids* các PID của các lệnh con khi thực thi. Sau đó, đem tách *args* thành các mảng con (là các lệnh hoàn chỉnh) tại mỗi ký tự

‘|’, lưu vào *cmds*. Tiếp tục duyệt lại *cmds* để kiểm tra chuyển hướng: nếu có lệnh nào không phải lệnh đầu tiên mà có ‘<’ hay không phải lệnh cuối mà có ‘>’ thì báo lỗi rồi trả về prompt.

Sau đó, tạo *prev_outpipe* chứa đầu đọc của pipe từ lệnh trước. Với mỗi lệnh con lưu trong *cmds*:

- + Nếu không phải lệnh cuối thì dùng *pipe()* tạo pipe mới.
- + Dùng *fork()* tạo tiến trình con, PID lưu vào *pids[]*. Ở tiến trình con: nếu không phải lệnh đầu, dùng *dup2()* lấy *prev_outpipe* làm đầu vào cho lệnh; nếu không phải lệnh cuối, dùng *dup2()* lấy đầu ghi của lệnh làm đầu ra; còn lại ở lệnh đầu và cuối thì cho phép dùng *handle_io_redirects()* chuyển hướng đầu vào/đầu ra, tuy nhiên lệnh đầu chỉ được phép chuyển hướng đầu vào, lệnh cuối chỉ được phép chuyển hướng lệnh ra. Sau đó, đóng các đường không cần thiết rồi thực thi lệnh bằng *execvp()*.
- + Ở tiến trình cha, đóng *prev_outpipe* và đầu ghi của pipe, lấy đầu vào của pipe gán vào *prev_outpipe* để sử dụng trong lệnh sau, dùng *waitpid()* chờ cho toàn bộ tiến trình con kết thúc, giải phóng bộ nhớ đã cấp phát cho *cmds* và *pids* rồi trả về prompt.

5. Kết thúc lệnh đang thực thi

- **Phương pháp:** tiến trình cha bắt SIGINT, khi nhận được thì tiến trình cha in ra prompt mới. Tiến trình con nhận được SIGINT thì kết thúc theo mặc định.
- **Minh chứng:**

The screenshot shows a terminal window with the following content:

```

GiB Mem : 36.0/23.2 [  ] GiB Swap: 0.0/24.0 [  ]

```

PID	USER	PR	NI	VIRT	RES	%CPU	%MEM	TIME+	S	COMMAND
1	root	20	0	65.9m	2.6m	0.0	0.0	0:01.49	S	/sbin/init
998	root	20	0	84.4m	6.5m	0.0	0.0	0:00.06	S	`- /usr/lib/systemd/sys+
1423	root	20	0	65.8m	2.6m	0.0	0.0	0:03.64	S	`- /usr/sbin/acpid
1726	message+	20	0	67.1m	6.2m	0.0	0.0	0:21.26	S	`- /usr/bin/dbus-daemon+
1756	root	20	0	87.6m	19.5m	0.0	0.1	0:31.81	S	`- /usr/sbin/NetworkMan+
1806	root	20	0	75.1m	8.4m	0.0	0.0	0:01.17	S	`- /usr/sbin/wpa_suppli+

it007sh> |

- **Giải thích:** ở hàm main, dùng `signal(SIGINT, signal_handler)` chờ bắt Ctrl-C, khi bắt được thì in xuống dòng, rồi vòng lặp tiếp tục in prompt như trước. Trong bước thực thi lệnh, đặt `signal(SIGINT, SIG_DFL)` trước `execvp()`. Khi nhận Ctrl-C, các tiến trình con đang chạy sẽ bị terminate, do hành vi mặc định là kết thúc tiến trình.